

Do we really need to recalibrate or retrain a brain-to-text decoder every day?

The decoder can stay frozen: recency-aware input-state management

Ron Nuriel

Itzhak Fried, Doron Friedman, Rafael Malach, Eilon Vaadia, Sarel Cohen

previous → geometry retrieval → calibration only if needed

Daily recalibration is a user burden

In BCI, this is not a theoretical question.

1

User time

Every calibration block is time the user spends maintaining the system.

2

Labeled data

Supervised updates need known prompts or labels at the start of the day.

3

Long-term usability

A helpful BCI should not become hard to use because it needs daily tuning.

So the practical question is not only “can we retrain?”

Can we keep the decoder fixed and manage the input state instead?

Reframe the daily problem

Not “train every day”; first ask which historical input state fits today.

Old question

How do we retrain
the model today?



New question

Which historical input state
is compatible with today's brain
signal?

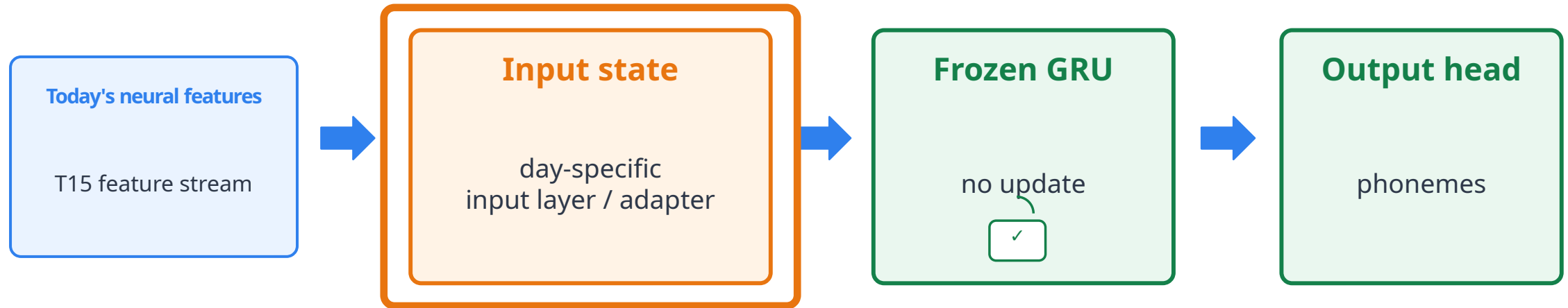
Main finding

**The decoder can stay frozen.
But the input state must be chosen.**

input state = day-specific input layer / adapter

Same frozen GRU. Different input adapter.

The GRU knows how to decode; the neural signal must enter through the right adapter.



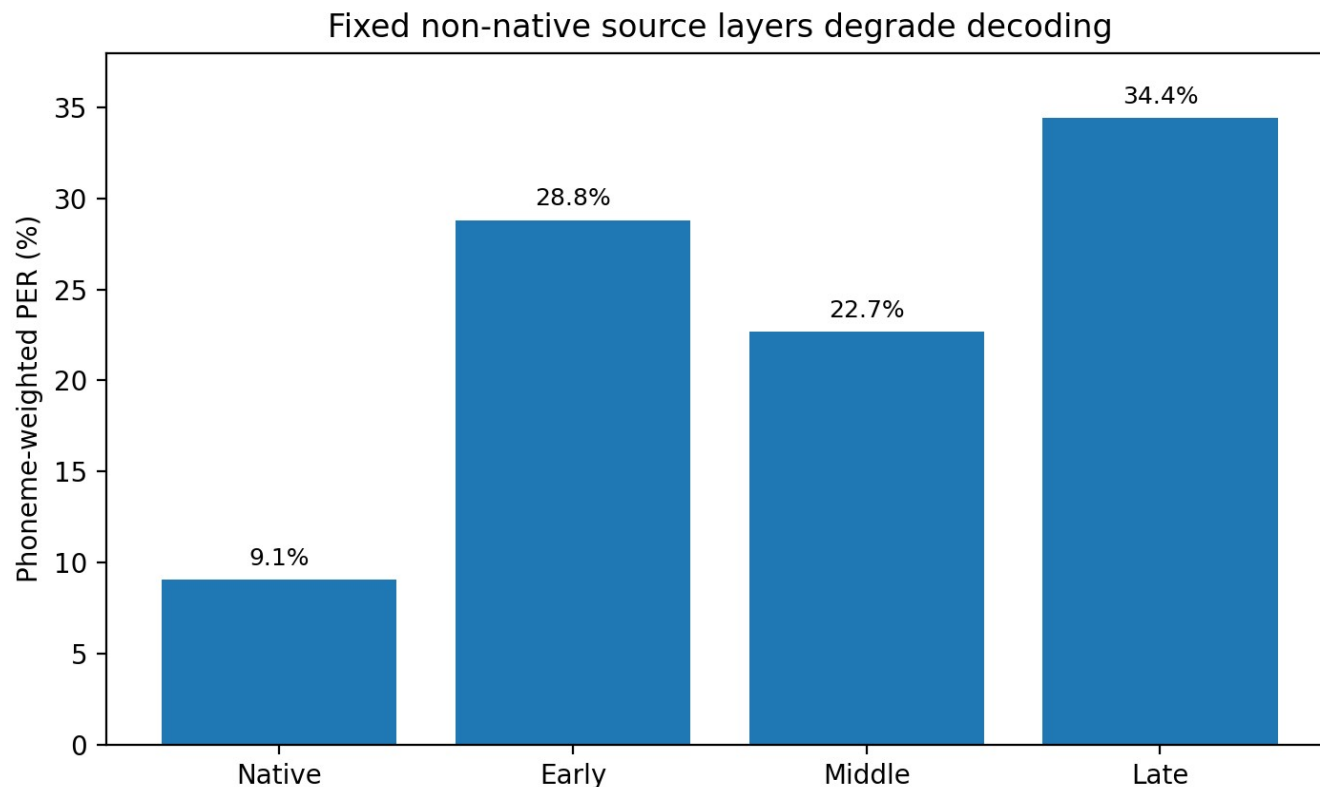
What changes?

Only the interface into the frozen decoder.

This is why we call the problem input-state management.

Result 1: wrong input-state selection breaks decoding

The decoder is not necessarily the problem — the interface into it is.



9.06%

native-day

correct daily input state

22–34%

wrong state

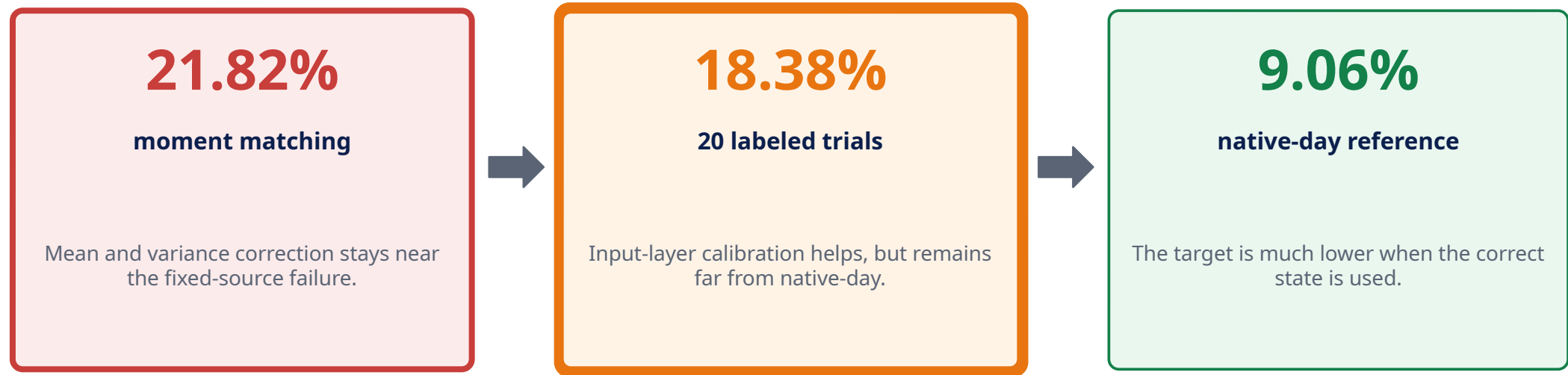
fixed non-native input
layers

**Same GRU. Different input state.
Huge PER jump.**

PER = phoneme error rate; lower is better.

Simple correction is too weak; small calibration is partial

Mean/variance matching does not reproduce the right daily input layer.



Calibration is useful — but it should not be the first move if a compatible state already exists.

Best low-cost move: select a compatible past input state

Before retraining, check whether the right adapter already exists in history.

12.21%

previous input state

Strongest simple policy at K = 20. No labels, no training.

12.74%

geometry retrieval

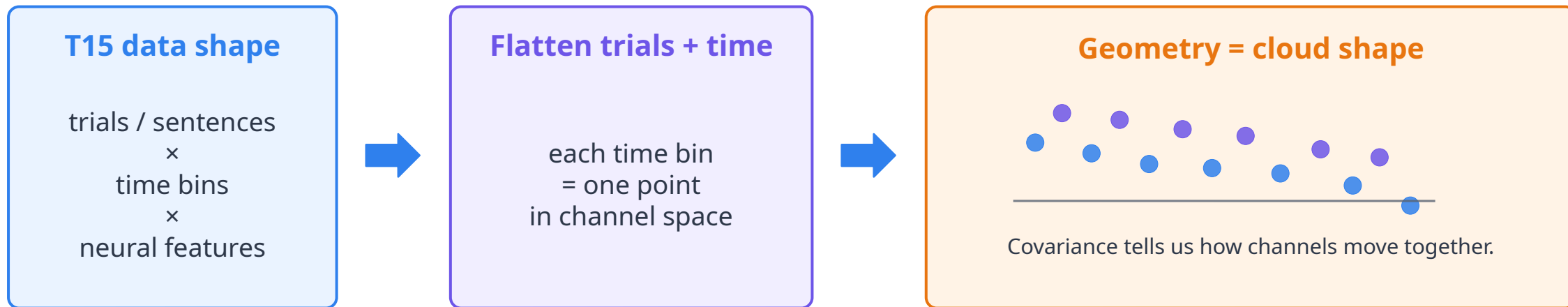
Close to previous; retrieves candidate states, but needs a threshold.

This is the core ladder:

reuse previous → retrieve by geometry → calibrate only if needed

What do we mean by “neural geometry”?

Each session becomes a cloud of neural-feature vectors.



If today's cloud shape resembles a past day, that past input layer may still be compatible.

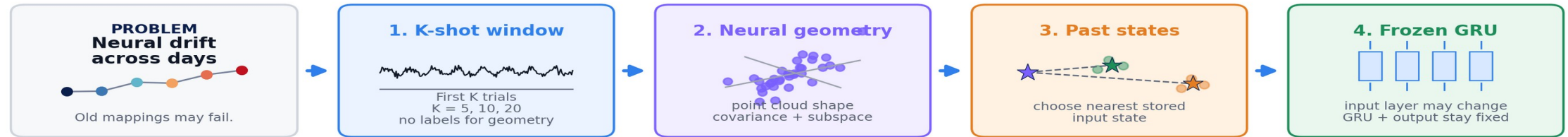
Uses a short unlabeled K-shot window: K = 5, 10, or 20 trials

The adaptation ladder: start cheap, escalate only when needed

Figure 1 from the paper — the whole method in one picture.

Recency-aware adaptation ladder for cross-session BCI drift

Estimate new-session geometry → retrieve a compatible input state → decode with the GRU frozen



Decision: start with the cheapest reliable action



Geometry: session as a cloud of neural-feature vectors; compare cloud shape by covariance / subspace.

Main distance:
$$d_{\text{cov}}(t, s) = \frac{\|\text{Cov}_t - \text{Cov}_s\|_F}{\|\text{Cov}_s\|_F}$$

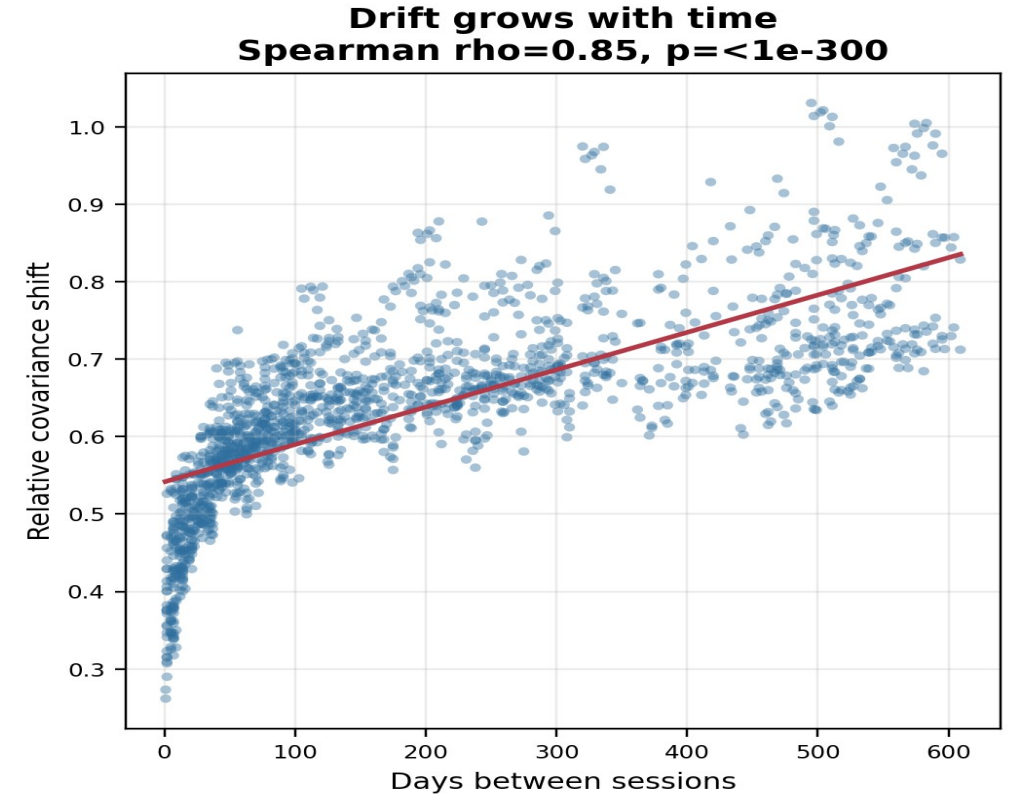
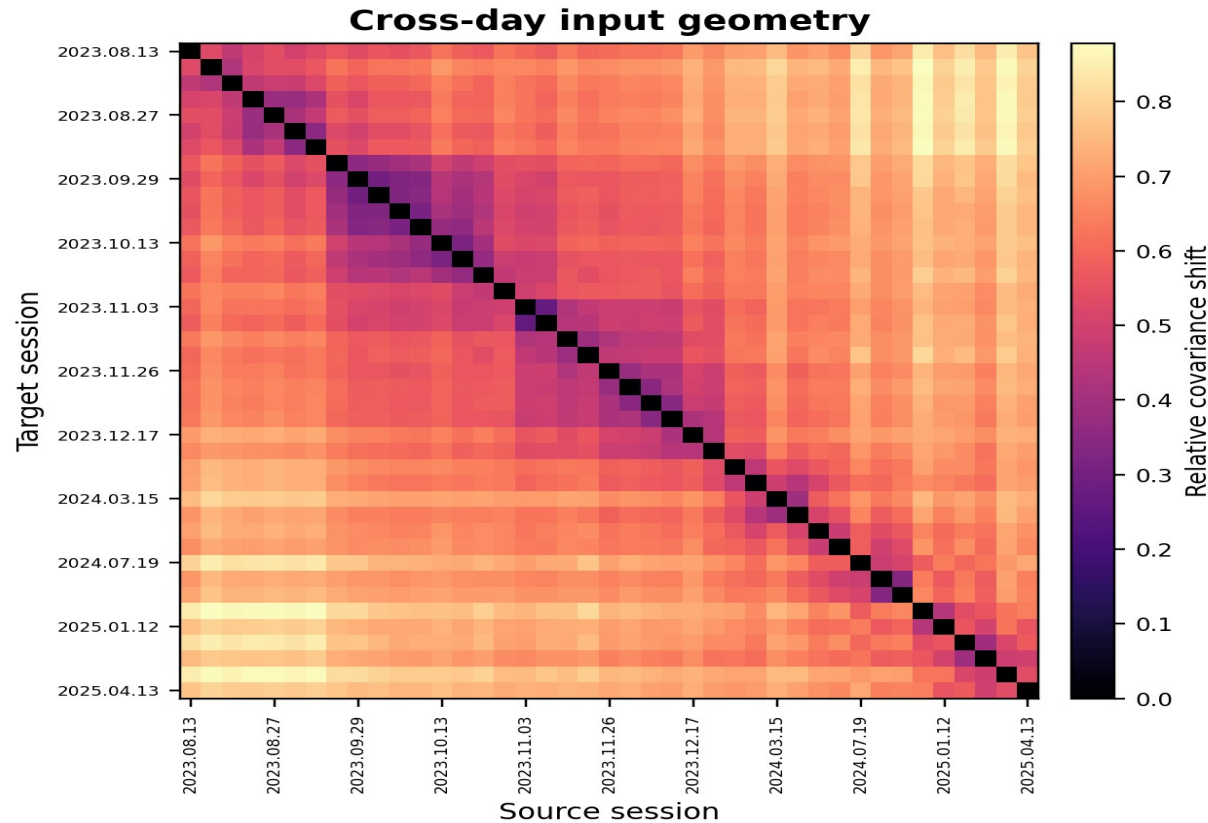
Take-home: previous is the default; geometry proposes older candidates; calibration is the last step.

Previous is default. Geometry retrieves candidates. Calibration is last.

Result 2: drift is structured, not random

Nearby days are often nearby in geometry — but the trajectory is not perfectly monotonic.

T15 longitudinal geometry: days close in time are usually close in input-state geometry



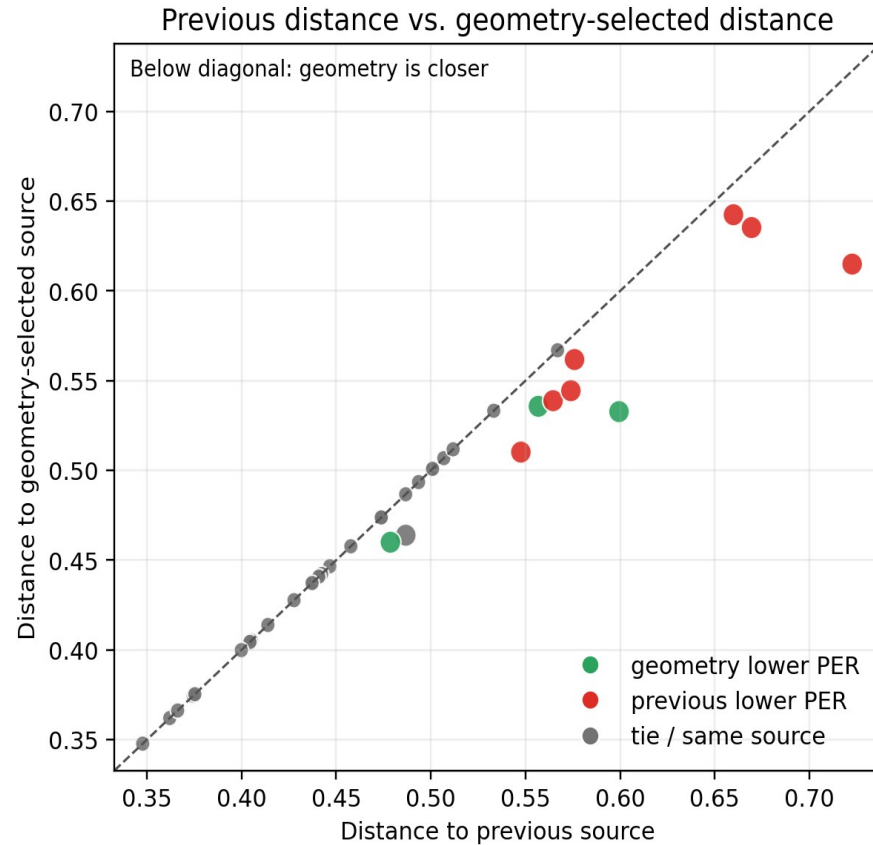
diagonal band = nearby days have similar geometry

Spearman $\rho = 0.85$ = drift grows with time

non-monotonic cases = older states can matter

Result 3: geometry proposes candidates — it does not decide alone

Closer in covariance is not always lower PER. A confidence gate is needed.



26

selected previous

sessions

11

selected older

sessions

3/11

older wins

only three cases

7/11

previous still better

when older was selected

3.23 pp

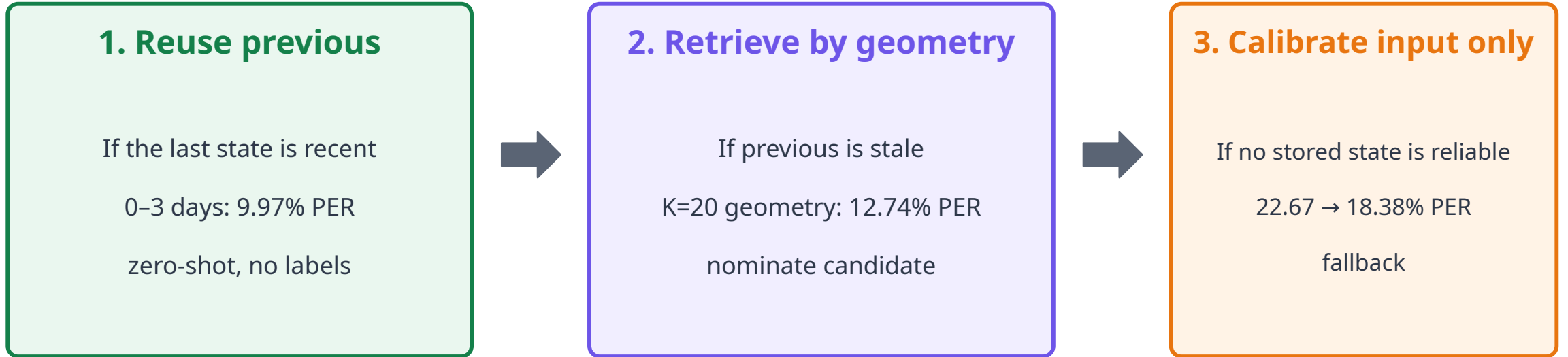
false-positive cost

average PER penalty

**Conclusion: geometry should nominate older states,
not blindly override recency.**

Operational answer: manage input states, not the decoder

Reuse → retrieve → calibrate.



The decoder can stay frozen; the system selects or lightly adapts the input state.

Next: confidence threshold, WER + LM decoding, and closed-loop validation

The decoder can stay frozen.

What needs to adapt is the input state.

Three takeaways

1

Wrong input states
break decoding

2

Drift has measurable
neural geometry

3

Geometry needs a
confidence threshold

Thank you